# REACT ROUTING – PRACTICAL ASSIGNMENTS

**Total Questions:** 10
**Difficulty Distribution:** Easy (2) | Medium (5) | Hard (3)

**Topics Covered:**

- BrowserRouter
- Routes and Route
- Link and NavLink
- useNavigate()
- useParams()
- useLocation()
- Dynamic Routing
- 404 Page
- Nested Routing

==========

# EASY LEVEL (2 QUESTIONS)

## Q1. Basic Multi-Page Navigation

Create a React application with the following pages:

1. Home
2. About
3. Contact
4. Services

Requirements:

- Create a separate component for each page.
- Configure all routes using BrowserRouter, Routes and Route.
- Create a navigation menu using Link.
- Clicking on any navigation link should display its respective component.
- The application should not reload when navigating between pages.
- Display a suitable heading on every page.

**Expected Result:** Users should be able to navigate between all four pages without refreshing the browser.

---

## Q2. Active Navigation Menu

Create a React application with the following navigation menu:

1. Home
2. Products

3. About
4. Contact

Requirements:

- Create separate components for all four pages.
- Use NavLink for navigation.
- Highlight the currently active navigation link using CSS.
- Use a different background color for the active link.
- The active styling should automatically change when the user navigates to another page.

**Expected Result:** Only the currently selected navigation item should appear active.

==========

# MEDIUM LEVEL (5 QUESTIONS)

## Q3. Login and Dashboard Navigation

Create a simple Login System with the following pages:

1. Login
2. Dashboard
3. Profile

Requirements:

- Create a Login page with Username and Password inputs.
- Add a Login button.
- When the user clicks Login, check whether both fields are filled.
- If both fields are filled, navigate to the Dashboard using useNavigate().
- If any field is empty, display an appropriate error message.
- Add a Profile button on the Dashboard.
- Clicking Profile should navigate to the Profile page.
- Add a Logout button that navigates back to Login.

**Expected Result:** Navigation should happen through button clicks and JavaScript logic instead of using Link for every action.

Note: This is a basic navigation exercise, not a secure authentication system.

---

## Q4. Student Details Using Dynamic Routing

Create a Student Management application with the following student information:

| Student ID | Student Name |
|---|---|
| 101 | Rahul |
| 102 | Priya |

| Student ID | Student Name |
|---|---|
| 103 | Amit |
| 104 | Neha |
| 105 | Karan |

Requirements:

- Create a Student List page.
- Display all five students with their IDs and names.
- Add a View Details link for every student.
- Use dynamic routing with the URL format `/student/:id`.
- Use useParams() to get the student ID from the URL.
- Display the corresponding student's ID and name on the Student Details page.
- If an invalid student ID is entered, display Student Not Found.

**Expected Result:** Every student should have a separate dynamic URL, but all students should use the same Student Details component.

---

## Q5. Product Details Application

Create a simple E-commerce Product Application.

Products:

1. Laptop – ₹55,000
2. Mobile – ₹25,000
3. Headphones – ₹2,000
4. Keyboard – ₹1,500
5. Mouse – ₹800

Requirements:

- Create a Product List page.
- Display all products with their names and prices.
- Add a View Product button for every product.
- Use dynamic routing with `/product/:id`.
- Use useParams() to get the product ID.
- Display the selected product's name and price.
- Add a Back to Products button.
- If the product ID does not exist, display Product Not Found.

**Expected Result:** Clicking on a product should open its individual details page.

---

## Q6. Current URL Information

Create a React application that displays information about the current URL.

Create the following pages:

1. Home
2. About
3. Services
4. Contact

Requirements:

- Create navigation using Link or NavLink.
- Use useLocation() to get the current location.
- Display the current pathname on every page.
- Display the current URL's search parameters if available.
- Display the current URL's hash if available.
- Add suitable links to test different routes.

**Expected Result:** Whenever the user navigates to another page, the displayed URL information should update automatically.

---

## Q7. 404 Page Not Found

Create a React application with the following pages:

1. Home
2. About
3. Services
4. Contact
5. Page Not Found

Requirements:

- Configure all valid routes.
- Create a separate NotFound component.
- Use the wildcard route * to handle invalid URLs.
- Display a 404 error message when the user visits an undefined route.
- Add a Back to Home button.
- Use useNavigate() to navigate back to the Home page.

**Expected Result:** Any URL that does not match the defined routes should display the custom 404 page.

==========

# HARD LEVEL (3 QUESTIONS)

---

## Q8. Nested Routing – Student Dashboard

Create a Student Dashboard application using Nested Routing.

Create the following structure:

Dashboard

- Overview
- My Courses
- My Results
- My Profile

Requirements:

- Create a Dashboard component.
- Create separate components for Overview, My Courses, My Results and My Profile.
- Configure nested routes using a parent Route and child Routes.
- Use Outlet to display the child components inside the Dashboard layout.
- Create a sidebar navigation using NavLink.
- Highlight the currently active sidebar link.
- When the user clicks on any sidebar item, only the dashboard content area should change.
- Keep the sidebar visible on every dashboard page.

**Expected Result:** The dashboard layout should remain fixed while its content changes according to the selected nested route.

---

## Q9. E-commerce Application with Dynamic Routing

Create a basic E-commerce application with the following pages:

1. Home
2. Product List
3. Product Details
4. Cart
5. About

Product Data:

| ID | Product | Price |
|---|---|---|
| 1 | Laptop | ₹55,000 |
| 2 | Mobile | ₹25,000 |
| 3 | Headphones | ₹2,000 |
| 4 | Keyboard | ₹1,500 |
| 5 | Mouse | ₹800 |

Requirements:

- Create navigation using NavLink.
- Display all products on the Product List page.
- Each product should have a View Details button.
- Use dynamic routing to display individual product details.

- Use useParams() to identify the selected product.
- Add an Add to Cart button on the Product Details page.
- After clicking Add to Cart, navigate to the Cart page using useNavigate().
- Display the selected product's name and price on the Cart page.
- Handle invalid product IDs.
- Add a 404 page for undefined routes.

**Expected Result:** Users should be able to navigate between pages, view individual products and add a selected product to the cart.

Note: Use a normal React variable or component state for cart data. No backend is required.

---

## Q10. Complete Course Management Dashboard

Create a Course Management application with the following structure:

Main Navigation:

1. Home
2. Courses
3. Dashboard
4. About

Dashboard Sidebar:

1. Overview
2. My Courses
3. My Profile
4. Settings

Requirements:

**Part A – Main Routing**

- Configure all main routes using BrowserRouter, Routes and Route.
- Use NavLink for the main navigation.
- Highlight the active navigation item.

**Part B – Nested Routing**

- Create a Dashboard component.
- Configure nested routes for Overview, My Courses, My Profile and Settings.
- Use Outlet to display the selected dashboard component.
- Keep the sidebar visible while navigating between dashboard sections.

**Part C – Dynamic Routing**

- Create at least five courses.
- Each course should have a unique ID, course name and duration.
- Use dynamic routing with `/courses/:id`.
- Use useParams() to display the selected course details.

- Display Course Not Found for invalid IDs.

**Part D – Programmatic Navigation**

- Add a View Course button to each course.
- Use useNavigate() to navigate to the selected course details.
- Add a Back to Courses button.

**Part E – URL Information and Error Handling**

- Use useLocation() to display the current pathname.
- Create a custom 404 page.
- Add a Back to Home button on the 404 page.

**Expected Result:** Build a complete multi-page Course Management application that combines normal routing, nested routing, dynamic routing and programmatic navigation.

==========

# GENERAL INSTRUCTIONS

1. Use React Router DOM for all routing tasks.
2. Create separate components wherever required.
3. Maintain a proper folder structure.
4. Use meaningful component names.
5. Use CSS to make the application presentable.
6. Do not reload the browser for normal navigation.
7. Test all routes manually.
8. Handle invalid URLs wherever required.
9. Use React hooks only where required by the task.

**Restrictions:**

- No backend is required.
- No database is required.
- Use static data wherever necessary.
- Focus mainly on routing concepts and component navigation.

# END OF ASSIGNMENTS